SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Name :_ K.JAYA MADHAVAN

Reg. No :**11239A053**

Class : **III B.E. (CSE) S1 Section**

Course Code: BCSF186P80

Course Name: **OPEN CV LAB**

SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that this is the bonafide record of work done by Mr/Ms. K.JAYA MADHAVAN, with Reg. No 11239A053 of III-B.E.(CSE-S1) during the academic year 2025 – 2026.

Station: Enathur

Date: 30-04-2026

Staff-in-charge 30/04

Head of the Department

Submitted for the Practical examination held on 30-04-2026.

Examiner-1

Examiner-2

INDEX

SL.NO	Program Name	Date	Page no	Signature
1.	Basic image operations	19-01-2026		
2.	Simple operations for binary Image processing	19-01-2026		
3.	Program to read and display image	02-02-2026		
4.	Program to display properties of image	02-02-2026		
5.	Program to convert original image to grayscale	09-02-2026		
6.	Program to convert original image to HSV	09-02-2026		
7.	Program to add text on the image	23-02-2026		
8.	Program to draw different shapes	23-02-2026		
9.	Program to read and display in various image format	02-03-2026		
10.	Program to rotate an image in various degrees	02-03-2026		
11.	Program to cropping an image	09-03-2026		
12.	Program to perform flipping of an image	09-03-2026		
13.	Program to perform smoothing and blurring	16-03-2026		
14.	Program to analyse an image using histogram	16-03-2026		
15.	Program to perform edge detection of an image	23-03-2026		

SL.NO	Program Name	Date	Page no	Signature
16.	Program to perform morphological operations	30-03-2026		
17.	Text detection and recognition using Tesseract	30-03-2026		

Ex-NO: 01 **PROGRAM TO PERFORM BASIC OPERATIONS**

DATE: 19-01-2026

AIM: To write a program to perform basic image operations.

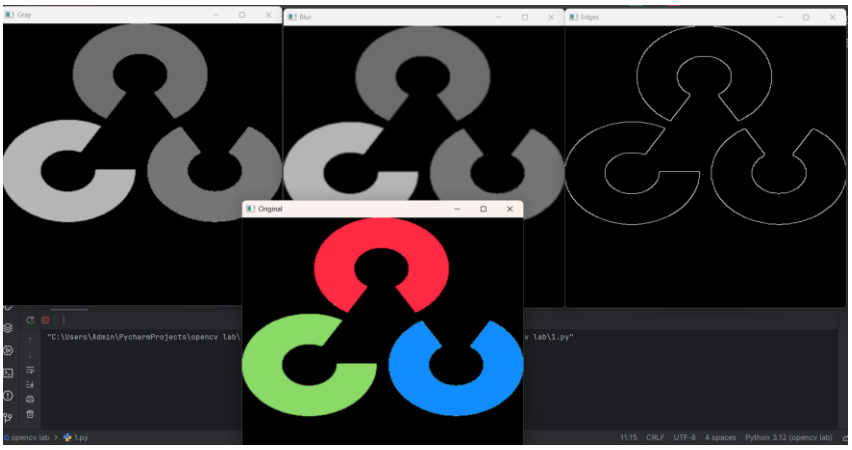
PROCEDURE:

1. Start the program and import OpenCV library.
2. Read the input image and resize it to a fixed size (500x500).
3. Convert the image to grayscale and apply Gaussian blur to reduce noise.
4. Detect edges in the image using Canny edge detection.
5. Display all processed images and wait for a key press to stop.

CODE:

```
import cv2
img = cv2.imread("img.png")
img = cv2.resize(img, (500, 500))
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5,5), 0)
edges = cv2.Canny(blur, 100, 200)
cv2.imshow("Original", img)
cv2.imshow("Gray", gray)
cv2.imshow("Blur", blur)
cv2.imshow("Edges", edges)
cv2.waitKey(0)
```

SAMPLE O/P:



RESULT: Thus the program to perform basic image operations is successfully executed.

Ex-NO.: 02 **SIMPLE OPERATIONS FOR BINARY IMAGE PROCESSING**

DATE: 19-01-2026

AIM: To write a program to perform simple operations for binary image processing.

PROCEDURE:

1. Start the program and import required libraries (OpenCV and NumPy**).**
2. Read the input image and convert it into a binary image using thresholding.
3. Create a structuring element (kernel) of size 5x5.
4. Apply morphological operations like erosion, dilation, opening, closing, gradient, tophat, and blackhat.
5. Display all the processed images and wait for a key press to stop.

CODE:

```
import cv2
import numpy as np
img=cv2.imread("img.png")
_,b=cv2.threshold(img,127,255,0)
k=np.ones((5,5),np.uint8)
cv2.imshow("binary",b)
cv2.imshow("erosion",cv2.erode(b,k))
cv2.imshow("dilation",cv2.dilate(b,k))
cv2.imshow("opening",cv2.morphologyEx(b,2,k))
cv2.imshow("closing",cv2.morphologyEx(b,3,k))
cv2.imshow("gradient",cv2.morphologyEx(b,4,k))
cv2.imshow("tophat",cv2.morphologyEx(b,5,k))
cv2.imshow("blackhat",cv2.morphologyEx(b,6,k))
cv2.waitKey(0)
```

SAMPLE O/P:

RESULT: Thus the program to perform simple operations for binary image processing is successfully executed.

Ex-NO.: 03 **PROGRAM TO READ AND DISPLAY IMAGE**
DATE: 02-02-2026

AIM: To write a program to read and display image.

PROCEDURE:

1. Start the program and import OpenCV library.
2. Read the input image from file.
3. Store the image in a variable.
4. Display the image in a window.
5. Wait for a key press to stop the program.

CODE:

```
import cv2
img=cv2.imread("img.png")
cv2.imshow("image:",img)
cv2.waitKey(0)
```

SAMPLE O/P:

RESULT: Thus the program to read and display image is successfully executed.

Ex-NO.:04 **DISPLAY PROPERTIES OF IMAGE**
DATE: 02-02-2026

AIM: To write a program to display properties of an image.

PROCEDURE:

1. Start the program and import OpenCV library**.**
2. Read the input image.
3. Extract image properties like height, width, channels, size, and datatype.
4. Store these values in variables.
5. Display height and width of the image.

CODE:

```
import cv2
img=cv2.imread("img.png")
height,width,channels=img.shape
size=img.size
datatype=img.dtype
print("height",height)
print("width",width)
```

SAMPLE O/P:

```
height 433
width 328
```

RESULT: Thus the program to display the properties of an image is successfully executed.

Ex-NO.: 05 **CONVERT ORIGINAL IMAGE TO GRAYSCALE**
DATE: 09-02-2026

AIM: To write a program to convert an image into grayscaleimage.

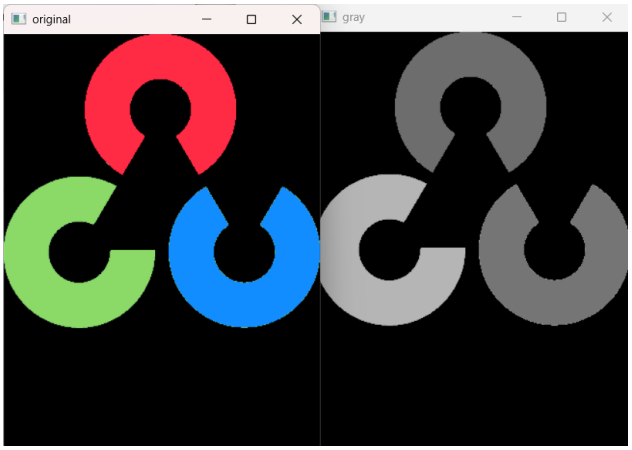
PROCEDURE:

1. Start the program and import OpenCV library.
2. Read the input image.
3. Convert the image from color to grayscale.
4. Display the grayscale image.
5. Wait for a key press to stop the program.

CODE:

```
import cv2
img=cv2.imread("img.png")
gray=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
cv2.imshow("gray",gray)
cv2.waitKey(0)
```

SAMPLE O/P:



RESULT: Thus the program to convert an image into grayscale image is successfully executed.

Ex-NO.: 06 **CONVERT IMAGE INTO HSV**
DATE: 09-02-2026

AIM: To write a program to convert an image into HSV image.

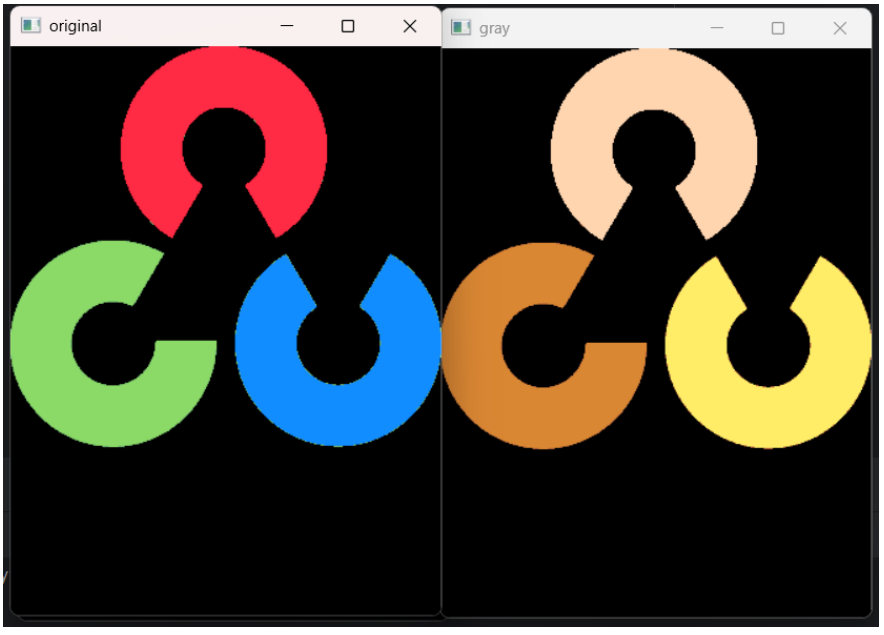
PROCEDURE:

1. Start the program and import OpenCV library.
2. Read the input image.
3. Convert the image from BGR color space to HSV color space.
4. Display the HSV image.
5. Wait for a key press to stop the program.

CODE:

```
import cv2
img=cv2.imread("img.png")
hsv=cv2.cvtColor(img,cv2.COLOR_BGR2HSV)
cv2.imshow("gray",hsv)
cv2.imshow("original",img)
cv2.waitKey(0)
```

SAMPLE O/P:



RESULT: Thus the program to convert image into HSV is successfully executed.

Ex-NO.:07 **ADD TEXT ON IMAGE**
DATE: 23-02-2026

AIM: To write a program to add text on an image.

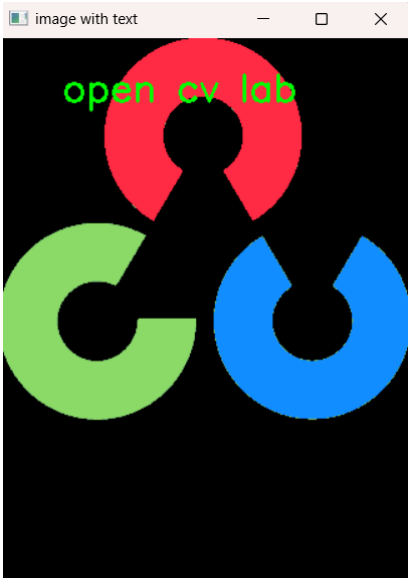
PROCEDURE:

1. Start the program and import OpenCV library.
2. Read the input image.
3. Add text to the image using putText function.
4. Display the image with text.
5. Wait for a key press to stop the program.

CODE:

```
import cv2
img=cv2.imread("img.png")
cv2.putText(img,"open cv lab",(50,50),cv2.FONT_HERSHEY_SIMPLEX,1,(0,255,0),2)
cv2.imshow("image with text",img)
cv2.waitKey(0)
```

SAMPLE O/P:



RESULT: Thus the program to add text on an image is successfully executed.

Ex-NO.: 08 **TO DRAW DIFFERENT SHAPES**
DATE: 23-02-2026

AIM: To write a program to draw different shapes.

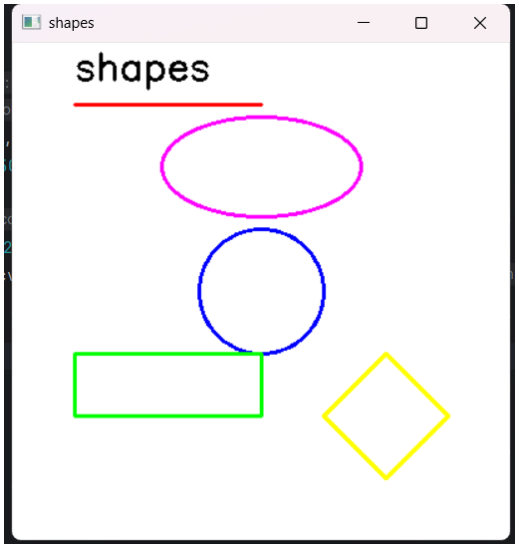
PROCEDURE:

1. Start the program and import OpenCV and NumPy libraries.
2. Create a blank image using NumPy.
3. Draw shapes like line, circle, rectangle, polygon, and ellipse on the image.
4. Add text to the image.
5. Display the final image and wait for a key press to stop.

CODE:

```
import cv2
import numpy as np
img=np.ones((400,400,3),np.uint8)*255
cv2.line(img,(50,50),(200,50),(0,0,255),2)
cv2.circle(img,(200,200),50,(255,0,0),2)
cv2.rectangle(img,(50,300),(200,250),(0,255,0),2)
pts=np.array([[250,300],[300,250],[350,300],[300,350]],np.int32)
pts=pts.reshape((-1,1,2))
cv2.polylines(img,[pts],True,(0,255,255),2)
cv2.ellipse(img,(200,100),(80,40),0,0,360,(255,0,255),2)
cv2.putText(img,"shapes",(50,30),cv2.FONT_HERSHEY_SIMPLEX,1,(0,0,0),2)
cv2.imshow("shapes",img)
cv2.waitKey(0)
```

SAMPLE O/P:



RESULT: Thus the program to draw different shapes is successfully executed.

Ex-NO.:09 **TO READ AND SIAPLY IMAG EIN DIFFERENT FORMAT**
DATE: 02-03-2026

AIM: To write a program to read and display an image in dufferent formats.

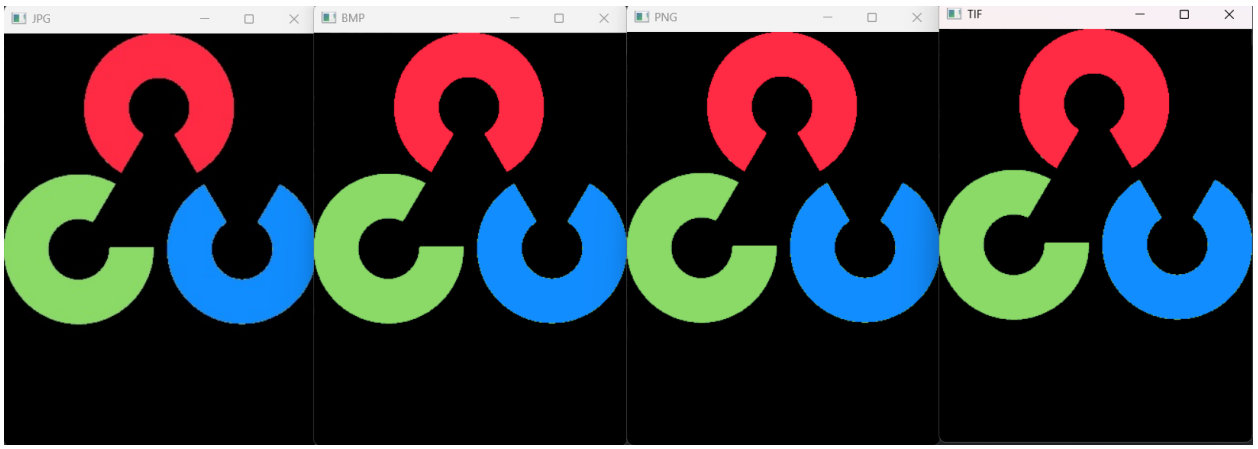
PROCEDURE:

1. Start the program and import OpenCV library.
2. Read the input image.
3. Save the image in different formats (PNG, BMP, JPG, TIF).
4. Read the saved images.
5. Display all images and wait for a key press to stop.

CODE:

```
import cv2
img=cv2.imread("img.png")
cv2.imwrite("output.png",img)
cv2.imwrite("output.bmp",img)
cv2.imwrite("output.jpg",img)
cv2.imwrite("output.tif",img)
png=cv2.imread("output.png")
bmp=cv2.imread("output.bmp")
jpg=cv2.imread("output.jpg")
tif=cv2.imread("output.tif")
cv2.imshow("PNG",png)
cv2.imshow("BMP",bmp)
cv2.imshow("JPG",jpg)
cv2.imshow("TIF",tif)
cv2.waitKey(0)
```

SAMPLE O/P:



RESULT: Thus the program to read and display an image in different formats is successfully executed.

Ex-NO.:10 ROTATE AN IMAGE IN DIFFERENT DEGREES
DATE: 02-03-2026

AIM: To write a program to rotate an image in different degrees..

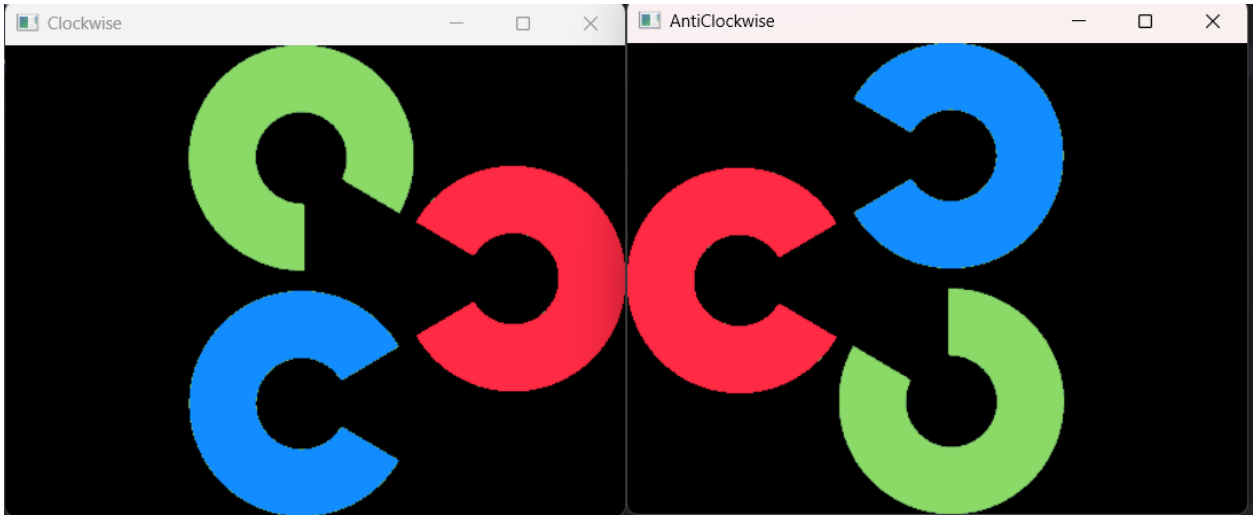
PROCEDURE:

1. Start the program and import OpenCV library.
2. Read the input image.
3. Rotate the image 90 degrees clockwise.
4. Rotate the image 90 degrees anticlockwise.
5. Display both rotated images and wait for a key press to stop.

CODE:

```
import cv2
img=cv2.imread("img.png")
cw=cv2.rotate(img,cv2.ROTATE_90_CLOCKWISE)
ccw=cv2.rotate(img,cv2.ROTATE_90_COUNTERCLOCKWISE)
cv2.imshow("Clockwise",cw)
cv2.imshow("AntiClockwise",ccw)
cv2.waitKey(0)
```

SAMPLE O/P:



RESULT: Thus, the program to rotate image into different degrees is succesfully executed.

Ex-NO.:11 **CROPPING AN IMAGE**
DATE: 09-03-2026

AIM: To write a program to crop an image.

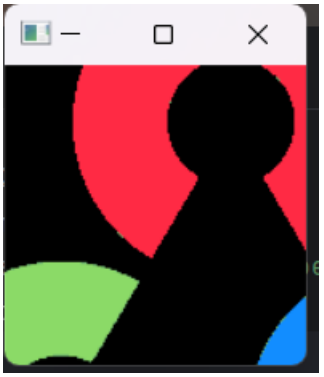
PROCEDURE:

1. Start the program and import OpenCV library.
2. Read the input image.
3. Select a region of interest (ROI) using slicing.
4. Store the cropped part of the image.
5. Display the cropped image and wait for a key press to stop.

CODE:

```
import cv2
img=cv2.imread("img.png")
crop=img[50:200,50:200]
cv2.imshow("cropped",crop)
cv2.waitKey(0)
```

SAMPLE O/P:



RESULT: Thus , the program to crop an image is successfully executed.

Ex-NO.: 12 **FLIPPING AN IMAGE**
DATE: 09-03-2026

AIM: To write a program to flip an image.

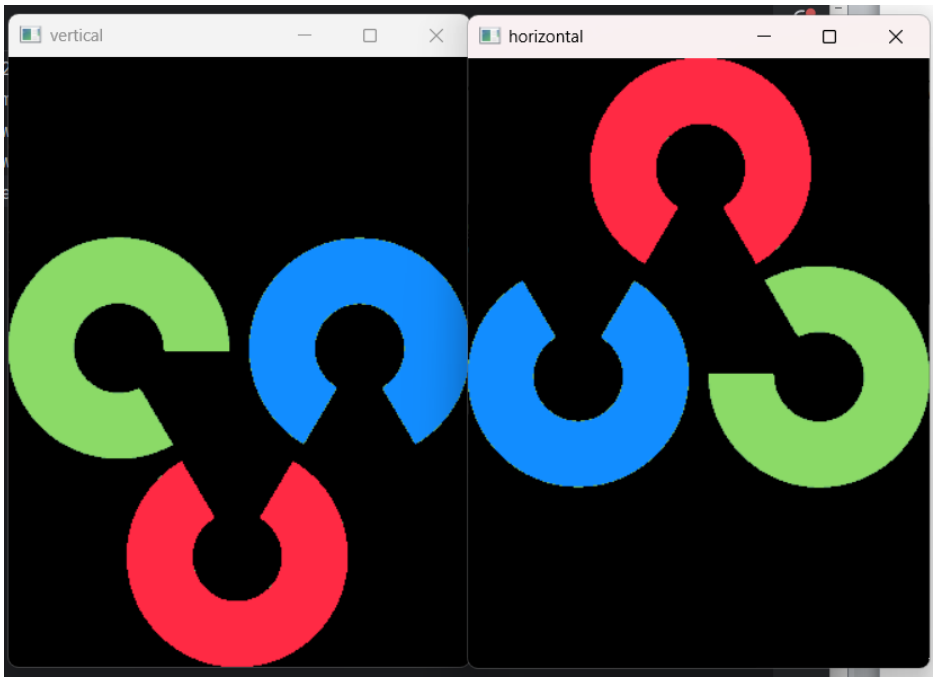
PROCEDURE:

1. Start the program and import OpenCV library.
2. Read the input image.
3. Flip the image vertically.
4. Flip the image horizontally.
5. Display both flipped images and wait for a key press to stop.

CODE:

```
import cv2
img=cv2.imread("img.png")
cv2.imshow("vertical",cv2.flip(img,0))
cv2.imshow("horizontal",cv2.flip(img,1))
cv2.waitKey(0)
```

SAMPLE O/P:



RESULT: Thus, the program to flip an image is successfully executed.

Ex-NO.: 13 **SMOOTHING AND BLURRING**
DATE: 16-03-2026

AIM: To write a program to perform smoothing and blurring.

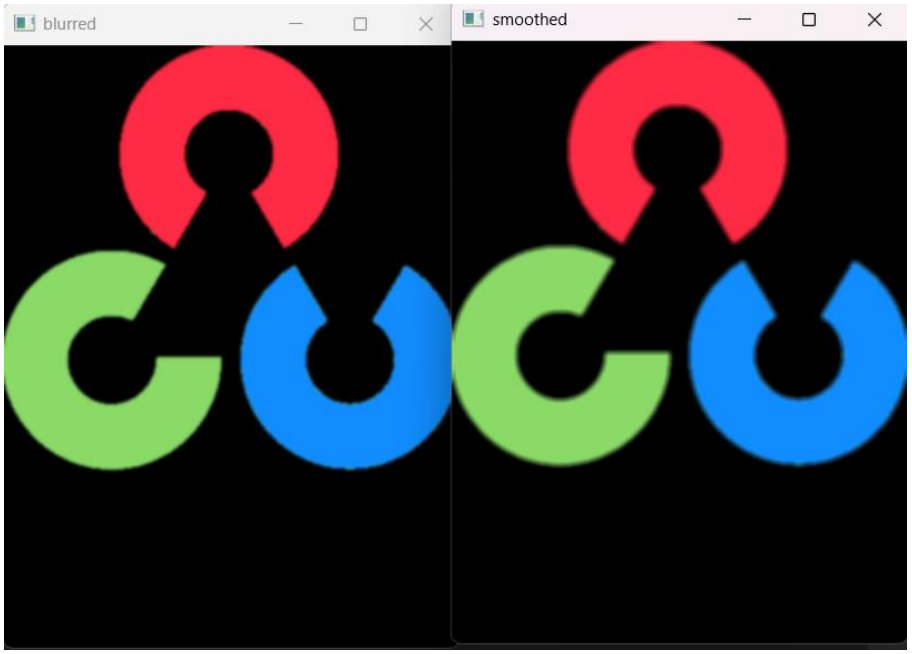
PROCEDURE:

1. Start the program and import OpenCV library.
2. Read the input image.
3. Apply Gaussian blur to the image.
4. Apply smoothing using average blur.
5. Display both processed images and wait for a key press to stop.

CODE:

```
import cv2
img=cv2.imread("img.png")
blur=cv2.GaussianBlur(img,(5,5),0)
smooth=cv2.blur(img,(5,5))
cv2.imshow("blurred",blur)
cv2.imshow("smoothed",smooth)
cv2.waitKey(0)
```

SAMPLE O/P:



RESULT: Thus the program for smoothing and blurring is successfully executed.

Ex-NO.:14 **ANALYSE IMAGE USING HISTOGRAM**
DATE: 16-03-2026

AIM: To write a program to analyse an image using histogram.

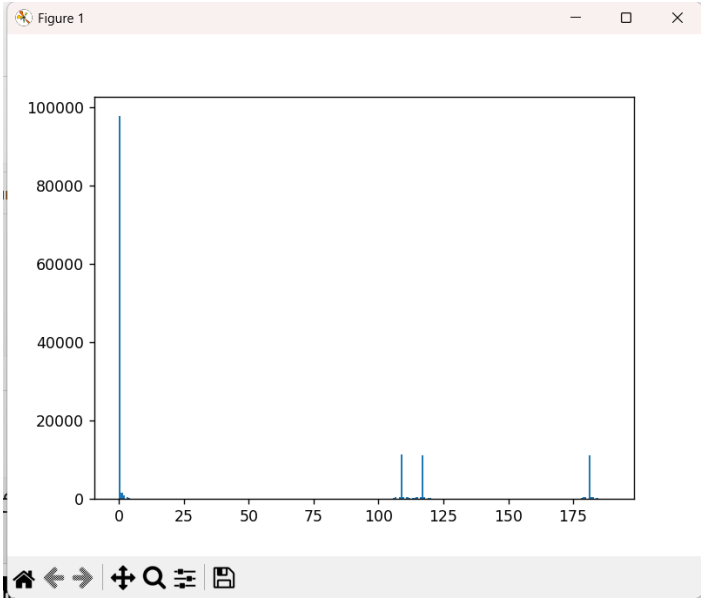
PROCEDURE:

1. Start the program and import OpenCV and Matplotlib libraries.
2. Read the input image in grayscale mode.
3. Convert the image into a 1D array using ravel().
4. Plot the histogram of pixel intensities.
5. Display the histogram graph.

CODE:

```
import cv2
import matplotlib.pyplot as plt
img=cv2.imread("img.jpg",0)
plt.hist(img.ravel(),256)
plt.show()
```

SAMPLE O/P:



RESULT: Thus the program to analyse an image using histogram is successfully executed

Ex-NO.: 15 **EDGE DETECTION OF AN IMAGE**
DATE: 23-02-2026

AIM: To write a program for edge detection of an image.

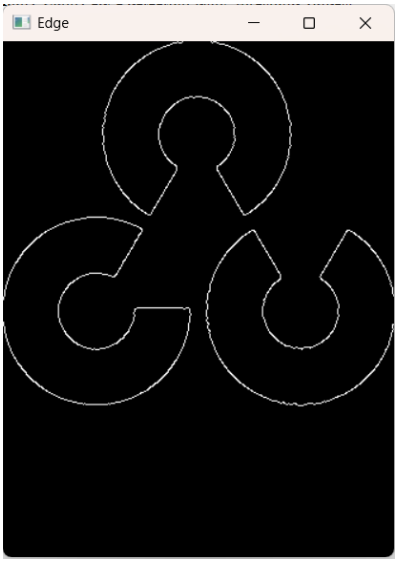
PROCEDURE:

1. Start the program and import OpenCV library.
2. Read the input image in grayscale mode.
3. Apply Canny edge detection using threshold values.
4. Store the detected edge image.
5. Display the edge image and wait for a key press to stop.

CODE:

```
import cv2
img=cv2.imread("img.jpg",0)
edge=cv2.Canny(img,100,200)
cv2.imshow("Edge",edge)
cv2.waitKey(0)
```

SAMPLE O/P:



RESULT: Thus, the program to demonstrate edge detection of an image is successfully executed.

Ex-NO.: 16 **MORPHOLOGICAL OPERATIONS**
DATE: 30-03-2026

AIM: To write a program to demonstrate morphological operations,

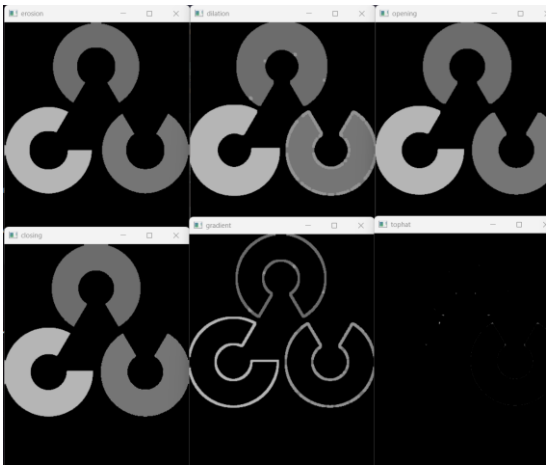
PROCEDURE:

1. Start the program and import OpenCV and NumPy libraries.
2. Read the input image in grayscale mode.
3. Create a structuring element (kernel) of size 5x5.
4. Apply morphological operations like erosion, dilation, opening, closing, gradient, tophat, and blackhat.
5. Display all processed images and wait for a key press to stop.

CODE:

```
import cv2
import numpy as np
img=cv2.imread("img.png",0)
k=np.ones((5,5),np.uint8)
cv2.imshow("erosion",cv2.erode(img,k))
cv2.imshow("dilation",cv2.dilate(img,k))
cv2.imshow("opening",cv2.morphologyEx(img,2,k))
cv2.imshow("closing",cv2.morphologyEx(img,3,k))
cv2.imshow("gradient",cv2.morphologyEx(img,4,k))
cv2.imshow("tophat",cv2.morphologyEx(img,5,k))
cv2.imshow("blackhat",cv2.morphologyEx(img,6,k))
cv2.waitKey(0)
```

SAMPLE O/P:



RESULT: Thus the program to demonstrate morphological operation sis successfully executed.

Ex-NO.: 17 **TEXT DETECTION AND RECOGNITION**
DATE: 30-03-2026

AIM: To write a program for text detection and recognition using tesseract.

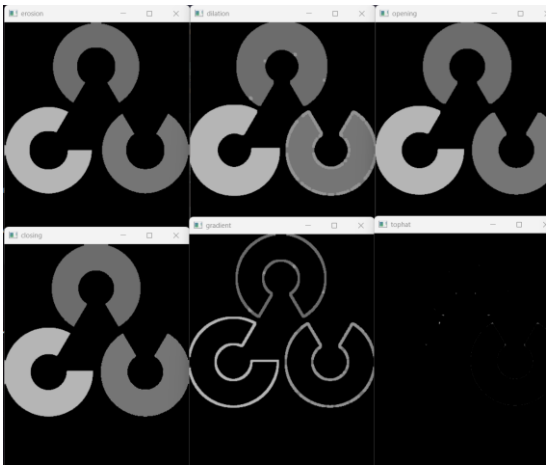
PROCEDURE:

1. Start the program and import OpenCV and pytesseract libraries.
2. Read the input image and check if it is loaded properly.
3. Convert the image to grayscale and apply thresholding.
4. Extract text from the processed image using OCR.
5. Display the text and images, then wait for a key press to stop.

CODE:

```
import cv2
import pytesseract
pytesseract.pytesseract.tesseract_cmd=r"C:\Program Files\Tesseract-OCR\tesseract.exe"
img=cv2.imread("test1.png")
if img is None:
    print("Image not found")
    exit()
gray=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
thresh=cv2.threshold(gray,150,255,cv2.THRESH_BINARY)[1]
text=pytesseract.image_to_string(thresh)
print("Detected Text:")
print(text)
cv2.imshow("Original",img)
cv2.imshow("Processed",thresh)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

SAMPLE O/P:



RESULT: Thus the program to demonstrate morphological operation sis successfully executed.